

	TRƯỜNG ĐẠI HỌC KỸ THUẬT - CÔNG NGHỆ TP.HCM (UTE) KHOA KỸ THUẬT MÁY TÍNH BÁO CÁO THỰC HÀNH HWSW CODESIGN – HSCD446164
---	--

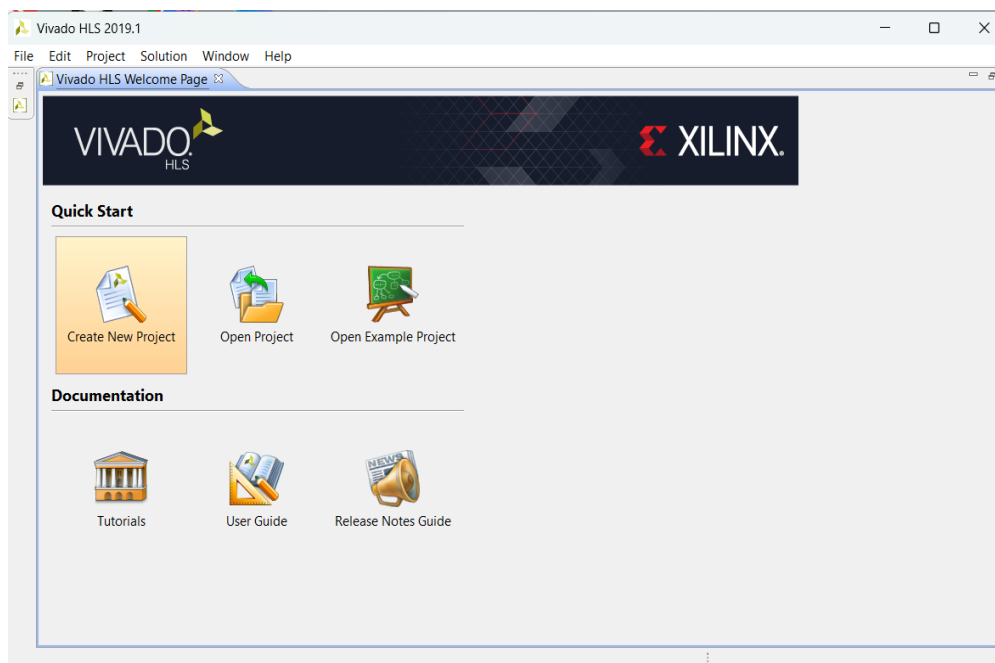
Nhóm	08
Thành viên	Trần Ngọc Dũng - 23119134 Huỳnh Chí Thiện - 23119207 Trần Chí Nguyên - 23119179 Nguyễn Hữu Trí - 22119244 Nguyễn Việt Nam - 23119173
Môn học	Thiết Kế Kết Hợp HW/SW - HSCD446164
Bài thực hành	Lab 3 – Thiết kế với Vivado HLS

PHẦN THỰC HÀNH 3A – TẠO PROJECT TRONG VIVADO HLS

Trong bài thực hành này, nhóm tiến hành tạo project Vivado HLS theo hai cách: sử dụng giao diện đồ họa (GUI) và sử dụng Tcl scripting. Mục đích là thiết lập môi trường để thực hiện tổng hợp phân cứng mức cao (HLS) cho hàm nhân ma trận `matrix_mult`.

Bước 1: Mở Vivado HLS và tạo project mới qua GUI

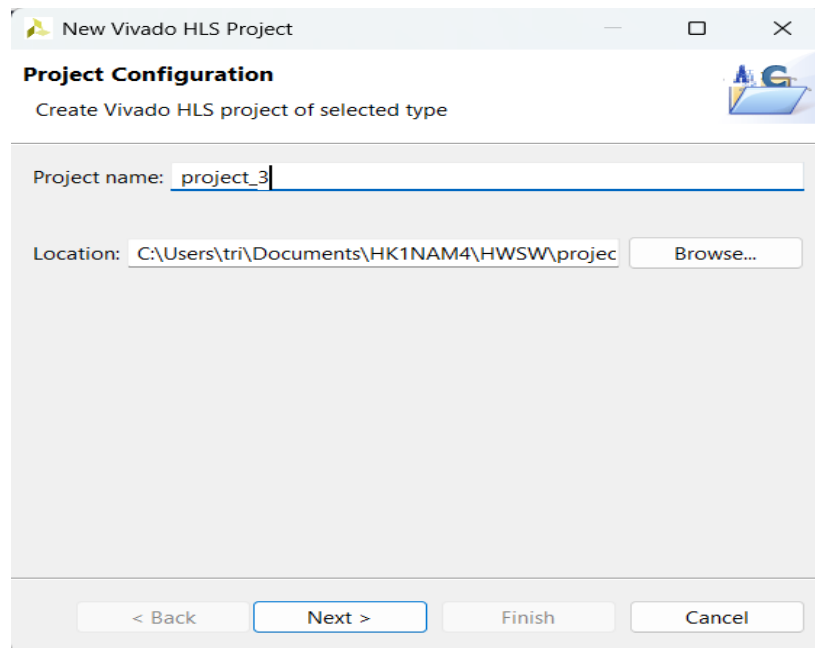
Khởi động Vivado HLS 2019.1. Màn hình Welcome Page xuất hiện với các lựa chọn: Create New Project, Open Project và Open Example Project. Nhóm chọn Create New Project để bắt đầu tạo project mới.



Hình 3A.1: Màn hình Welcome Page của Vivado HLS 2019.1

Bước 2: Cấu hình tên và đường dẫn project

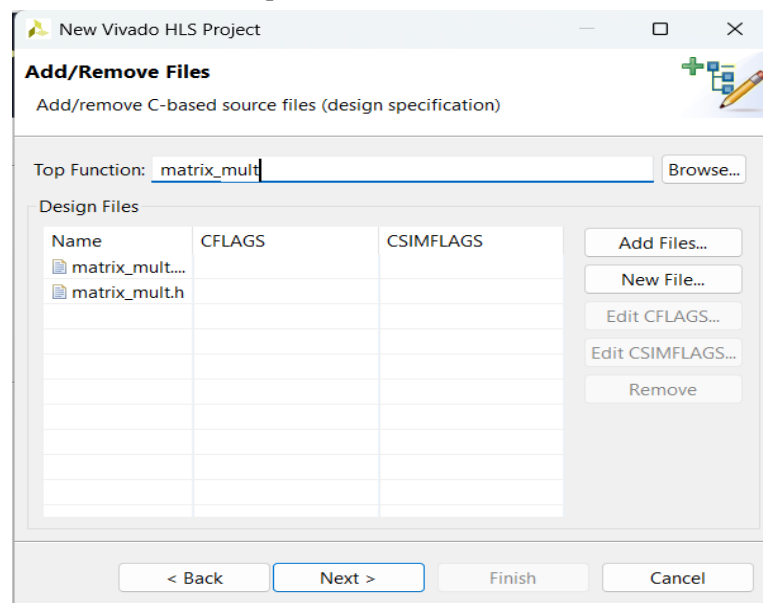
Trong hộp thoại Project Configuration, nhóm đặt tên project là "project_3" và chọn thư mục lưu trữ tại `C:\Users\tri\Documents\HK1NAM4\HWSW\project`. Sau đó nhấn Next để tiếp tục.



Hình 3A.2: Hộp thoại cấu hình tên và vị trí project

Bước 3: Thêm các file source

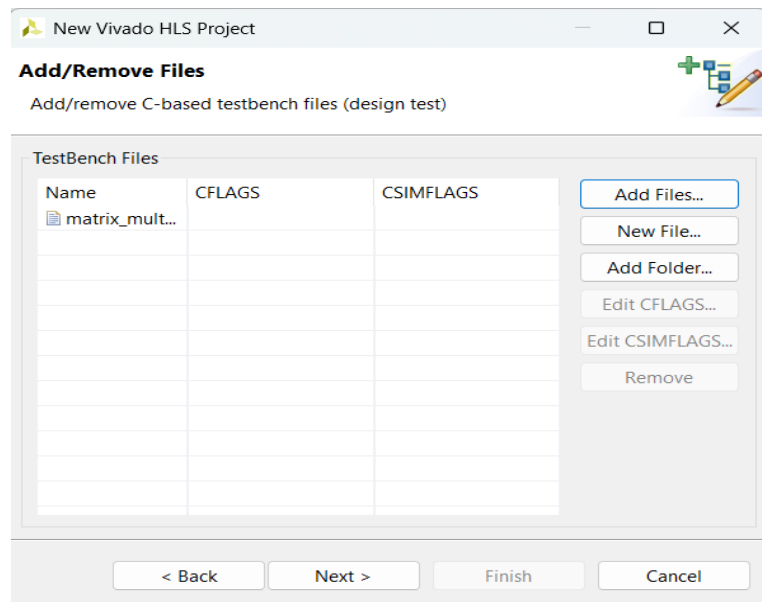
Trong bước Add/Remove Files (Design Specification), nhóm thêm hai file nguồn: `matrix_mult.cpp` (chứa code thực hiện phép nhân ma trận) và `matrix_mult.h` (chứa định nghĩa và prototype hàm). Top Function được đặt là `matrix_mult`. Nhấn Next để tiếp tục.



Hình 3A.3: Thêm file source `matrix_mult.cpp` và `matrix_mult.h`, đặt Top Function là `matrix_mult`

Bước 4: Thêm file testbench

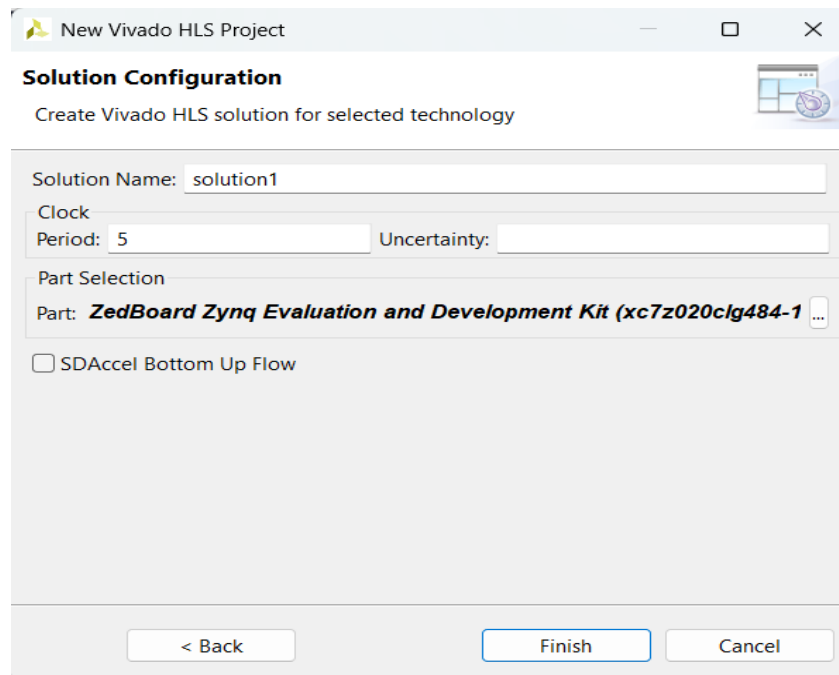
Trong bước Add/Remove Files (Design Test), nhóm thêm file `matrix_mult_test.cpp` làm testbench để kiểm tra tính đúng đắn của thiết kế. Nhấn Next để tiếp tục.



Hình 3A.4: Thêm file testbench matrix_mult_test.cpp

Bước 5: Cấu hình Solution và chọn thiết bị

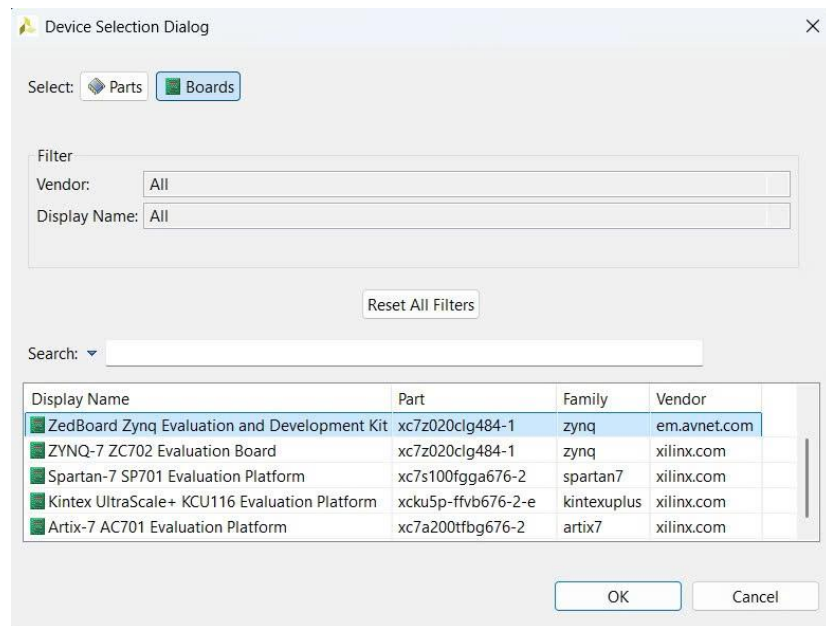
Trong bước Solution Configuration, đặt Solution Name là "solution1", Clock Period là 5 (ns). Ở mục Part Selection, chọn bo mạch ZedBoard Zynq Evaluation and Development Kit (chip xc7z020clg484-1) bằng cách nhấn nút "..." để mở Device Selection Dialog.



Hình 3A.5: Cấu hình Solution với clock period 5ns và chọn ZedBoard

Bước 6: Chọn thiết bị mục tiêu

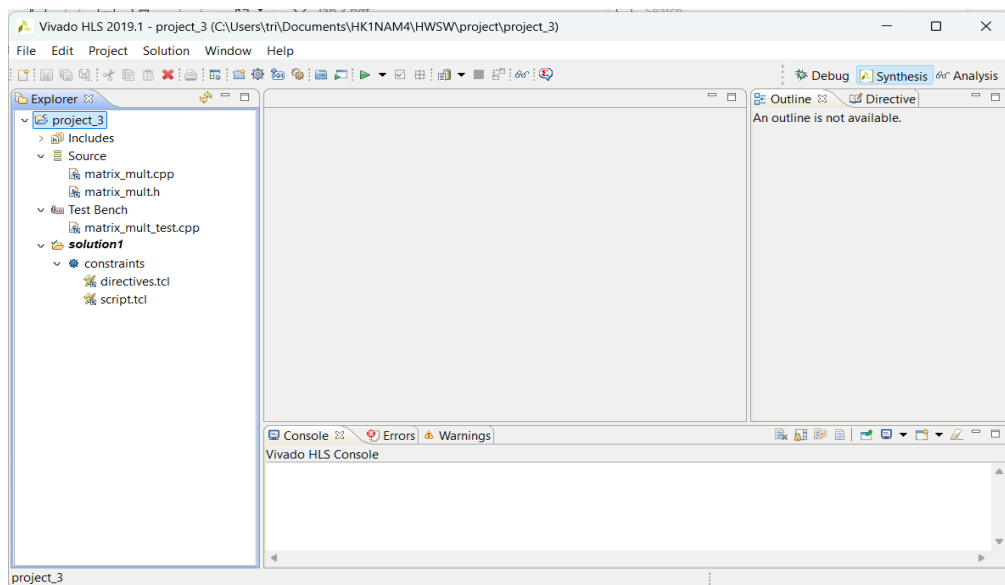
Trong Device Selection Dialog, chuyển sang tab Boards và lọc để tìm ZedBoard Zynq Evaluation and Development Kit (xc7z020clg484-1, family zynq, vendor em.avnet.com). Chọn board này rồi nhấn OK, sau đó nhấn Finish để hoàn tất tạo project.



Hình 3A.6: Hộp thoại Device Selection – chọn ZedBoard Zynq

Bước 7: Workspace project sau khi tạo

Project được tạo thành công. Trong tab Explorer bên trái, có thể thấy cấu trúc project với các thư mục Source (matrix_mult.cpp, matrix_mult.h), Test Bench (matrix_mult_test.cpp) và solution1 kèm file constraints. Workspace hiển thị ở chế độ Synthesis.



Hình 3A.7: Workspace Vivado HLS sau khi tạo project thành công

Bước 8: Tạo project bằng Tcl Script (Command Prompt)

Ngoài cách dùng GUI, nhóm còn tạo project bằng Tcl script. Mở Vivado HLS Command Prompt, thư mục mặc định là C:\Xilinx\Vivado\2019.1>.

```
Vivado HLS 2019.1 Command
=====
== Vivado HLS Command Prompt
== Available commands:
== vivado_hls, apcc, gcc, g++, make
=====
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Xilinx\Vivado\2019.1>
```

Hình 3A.8: Vivado HLS Command Prompt khởi động tại thư mục cài đặt

Bước 9: Chạy Tcl script tạo project và C Simulation

Di chuyển đến thư mục chứa source files bằng lệnh "cd

C:\Users\tri\Documents\HK1NAM4\HWSW\zynq-book-master\hls\tut3A", sau đó chạy lệnh "vivado_hls -f run_hls_zed.tcl". Script tự động tạo project matrix_mult_prj, thêm các file design và testbench, tạo solution1, đặt target device là xc7z020-clg484-1 với clock 5ns và chạy C Simulation. Kết quả "Test passed!" xác nhận simulation thành công.

```
Vivado HLS 2019.1 Command
=====
== Vivado HLS Command Prompt
== Available commands:
== vivado_hls, apcc, gcc, g++, make
=====
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Xilinx\Vivado\2019.1>cd C:\Users\tri\Documents\HK1NAM4\HWSW\zynq-book-master\hls\tut3A
C:\Users\tri\Documents\HK1NAM4\HWSW\zynq-book-master\hls\tut3A>vivado_hls -f run_hls_zed.tcl

***** Vivado(TM) HLS - High-Level Synthesis from C, C++ and SystemC v2019.1 (64-bit)
**** SW Build 2552052 on Fri May 24 14:49:42 MDT 2019
**** IP Build 2548770 on Fri May 24 18:01:18 MDT 2019
** Copyright 1986-2019 Xilinx, Inc. All Rights Reserved.

source C:/Xilinx/Vivado/2019.1/scripts/vivado_hls/hls.tcl -notrace
INFO: [HLS 200-10] Running 'C:/Xilinx/Vivado/2019.1/bin/unwrapped/win64.o/vivado_hls.exe'
INFO: [HLS 200-10] For user 'tri' on host 'laptop-4mve4bdb' (Windows NT_amd64 version 6.2) on Wed May 27 08:51:18 +0700 2026
INFO: [HLS 200-10] In directory 'C:/Users/tri/Documents/HK1NAM4/HWSW/zynq-book-master/hls/tut3A'
Sourcing Tcl script 'run_hls_zed.tcl'
INFO: [HLS 200-10] Creating and opening project 'C:/Users/tri/Documents/HK1NAM4/HWSW/zynq-book-master/hls/tut3A/matrix_mult_prj'
INFO: [HLS 200-10] Adding design file 'matrix_mult.cpp' to the project
INFO: [HLS 200-10] Adding design file 'matrix_mult.h' to the project
INFO: [HLS 200-10] Adding test bench file 'matrix_mult_test.cpp' to the project
INFO: [HLS 200-10] Creating and opening solution 'C:/Users/tri/Documents/HK1NAM4/HWSW/zynq-book-master/hls/tut3A/matrix_mult_prj/solution1'.
```

Hình 3A.9: Chạy lệnh vivado_hls -f run_hls_zed.tcl – quá trình tạo project

```
Vivado HLS 2019.1 Command
**** SW Build 2552052 on Fri May 24 14:49:42 MDT 2019
**** IP Build 2548770 on Fri May 24 18:01:18 MDT 2019
** Copyright 1986-2019 Xilinx, Inc. All Rights Reserved.

source C:/Xilinx/Vivado/2019.1/scripts/vivado_hls/hls.tcl -notrace
INFO: [HLS 200-10] Running 'C:/Xilinx/Vivado/2019.1/bin/unwrapped/win64.o/vivado_hls.exe'
INFO: [HLS 200-10] For user 'tri' on host 'laptop-4mve4bdb' (Windows NT_amd64 version 6.2) on Wed May 27 08:51:18 +0700 2026
INFO: [HLS 200-10] In directory 'C:/Users/tri/Documents/HK1NAM4/HWSW/zynq-book-master/hls/tut3A'
Sourcing Tcl script 'run_hls_zed.tcl'
INFO: [HLS 200-10] Creating and opening project 'C:/Users/tri/Documents/HK1NAM4/HWSW/zynq-book-master/hls/tut3A/matrix_mult_prj'
INFO: [HLS 200-10] Adding design file 'matrix_mult.cpp' to the project
INFO: [HLS 200-10] Adding design file 'matrix_mult.h' to the project
INFO: [HLS 200-10] Adding test bench file 'matrix_mult_test.cpp' to the project
INFO: [HLS 200-10] Creating and opening solution 'C:/Users/tri/Documents/HK1NAM4/HWSW/zynq-book-master/hls/tut3A/matrix_mult_prj/solution1'
INFO: [HLS 200-10] Cleaning up the solution database.
INFO: [HLS 200-10] Setting target device to 'xc7z020-clg484-1'
INFO: [SYN 201-201] Setting up clock 'default' with a period of 5ns.
INFO: [SIM 211-2] ***** CSIM start *****
INFO: [SIM 211-4] CSIM will launch GCC as the compiler.
Compiling ../../../../matrix_mult_test.cpp in debug mode
Compiling ../../../../matrix_mult.cpp in debug mode
Generating csim.exe
Test passed!
INFO: [SIM 211-1] CSim done with 0 errors.
INFO: [SIM 211-3] ***** CSIM finish *****
INFO: [Common 17-206] Exiting vivado_hls at Wed May 27 08:51:29 2026...
C:\Users\tri\Documents\HK1NAM4\HWSW\zynq-book-master\hls\tut3A>
```

Hình 3A.10: Kết quả chạy Tcl script – "Test passed!" xác nhận C Simulation thành công

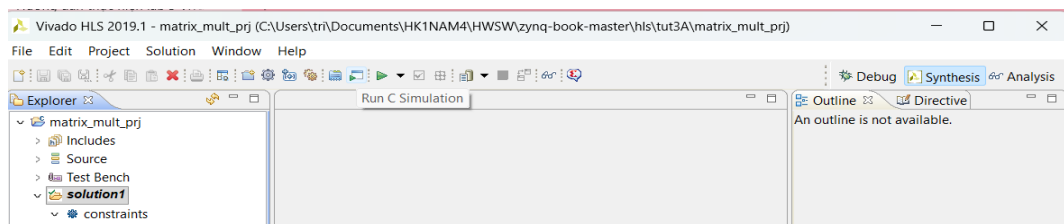
PHẦN THỰC HÀNH 3B – TỐI ƯU HÓA THIẾT KẾ TRONG VIVADO HLS

Trong phần này, nhóm thực hiện tối ưu hóa thiết kế nhân ma trận qua nhiều solution khác nhau, sử dụng các directive như PIPELINE và ARRAY_RESHAPE để cải thiện hiệu năng. Quá trình bao gồm C Simulation, C Synthesis, C/RTL Cosimulation và phân tích kết quả.

Bước 1: Mở project matrix_mult_prj và chạy C Simulation

Mở project matrix_mult_prj (đã tạo bằng Tcl script ở phần 3A) bằng câu lệnh: vivado_hls -p matrix_mult_prj

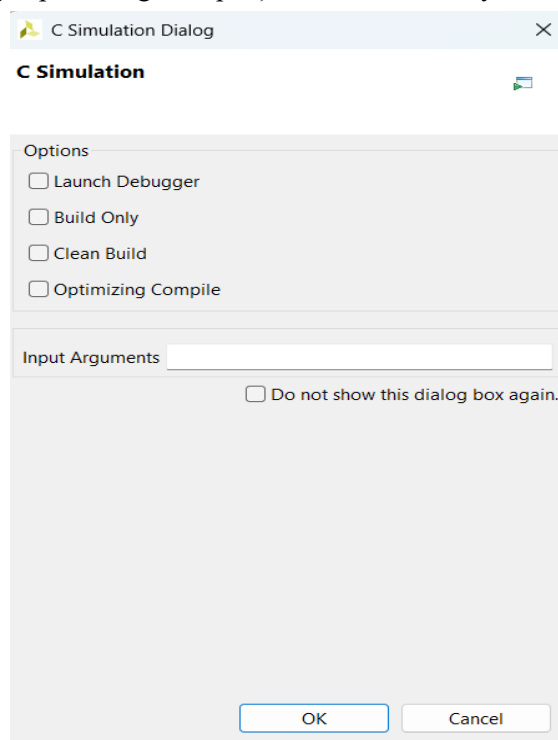
Nhấn nút "Run C Simulation" trên toolbar để chạy mô phỏng phần mềm.



Hình 3B.1: Nút Run C Simulation trên toolbar

Bước 2: Hộp thoại C Simulation

Trong hộp thoại C Simulation Dialog, giữ nguyên các tùy chọn mặc định (không chọn Launch Debugger, Build Only, Clean Build, hay Optimizing Compile). Nhấn OK để chạy.



Hình 3B.2: Hộp thoại C Simulation với các tùy chọn mặc định

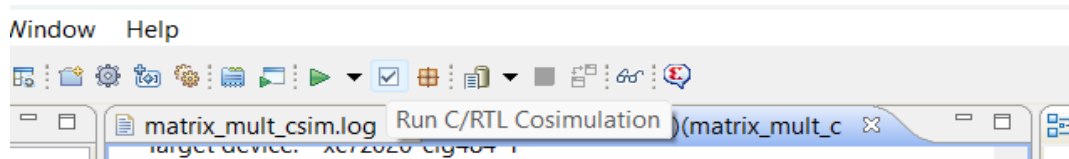
Bước 3: Kết quả C Simulation – Solution 1

Kết quả C Simulation hiển thị trong Console. Thông báo "Test passed!" xác nhận rằng hàm matrix_mult hoạt động đúng. CSim done with 0 errors cho thấy không có lỗi nào xảy ra trong quá trình mô phỏng.

Hình 3B.5: Synthesis Report solution1 – Latency 686 cycles, chưa pipeline

Bước 6: Chạy C/RTL Cosimulation – Solution 1

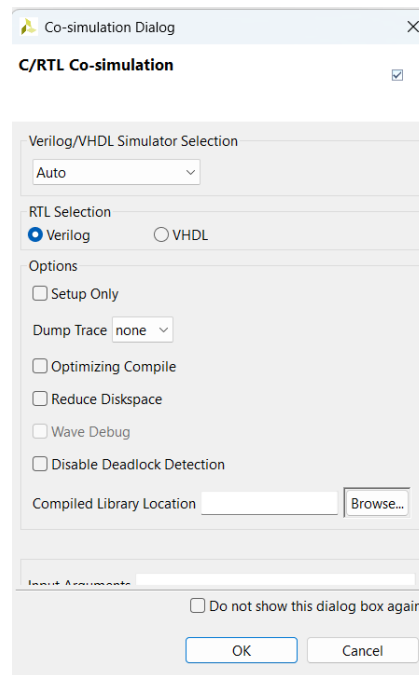
Nhấn nút "Run C/RTL Cosimulation" trên toolbar để kiểm tra RTL có hành vi giống với C++ không.



Hình 3B.6: Nút Run C/RTL Cosimulation trên toolbar

Bước 7: Cấu hình Cosimulation

Trong hộp thoại Co-simulation Dialog, chọn RTL Selection là Verilog, giữ các tùy chọn khác mặc định. Nhấn OK để bắt đầu cosimulation.



Hình 3B.7: Hộp thoại C/RTL Co-simulation – chọn Verilog

Bước 8: Kết quả Cosimulation Report – Solution 1

Cosimulation Report hiển thị kết quả: Verilog – Status: Pass, Latency min/avg/max đều là 686 cycles, Interval NA. Kết quả Pass xác nhận RTL tổng hợp có hành vi hoàn toàn giống với code C++ gốc.

Cosimulation Report for 'matrix_mult'

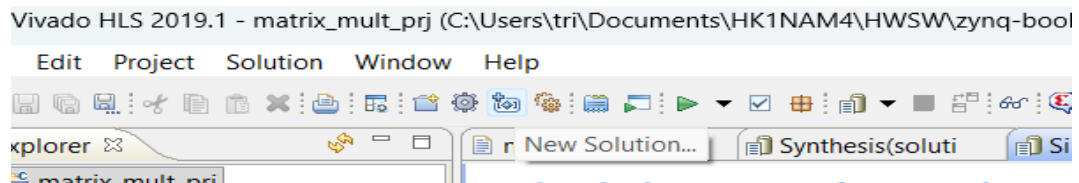
Result							
RTL	Status	Latency			Interval		
		min	avg	max	min	avg	max
VHDL	NA	686	686	686	NA	NA	NA
Verilog	Pass	686	686	686	NA	NA	NA

Export the report(.html) using the [Export Wizard](#)

Hình 3B.8: Cosimulation Report solution1 – Verilog Pass với latency 686 cycles

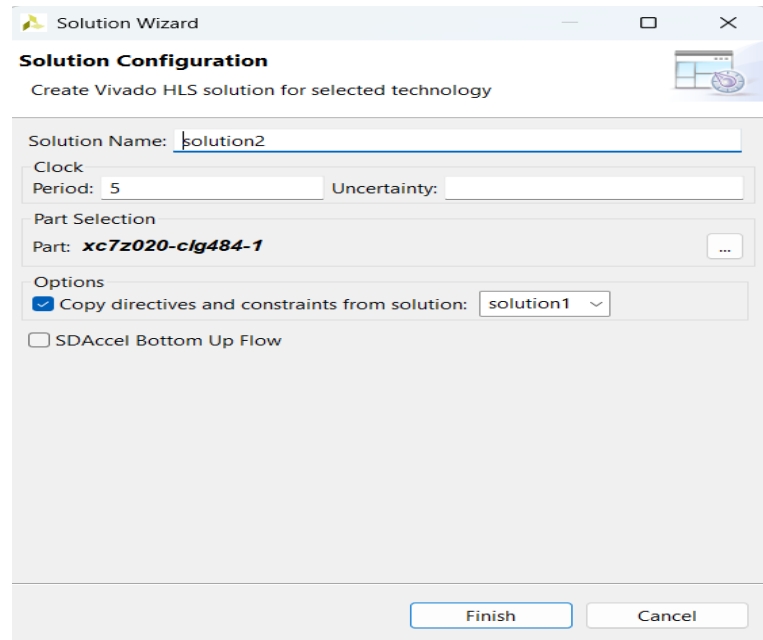
Bước 9: Tạo Solution 2 – Pipeline vòng lặp Product

Tạo solution mới bằng cách nhấn nút "New Solution" trên toolbar.



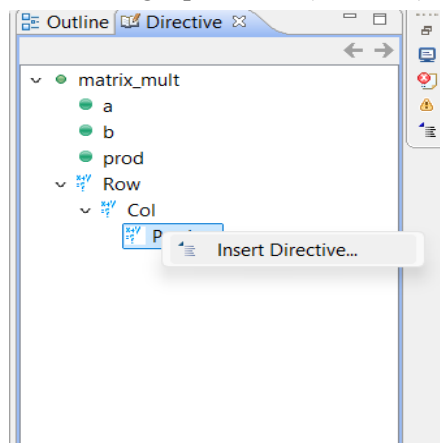
Hình 3B.9: Nút New Solution trên toolbar

Trong Solution Wizard, đặt tên là "solution2", clock period 5ns, part xc7z020-clg484-1, copy directives từ solution1. Nhấn Finish để tạo.



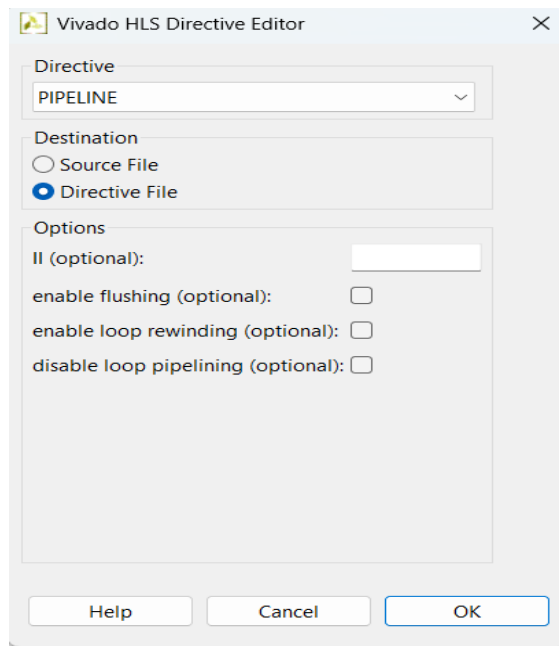
Hình 3B.10: Solution Wizard tạo solution2

Trong tab Directive, click chuột phải vào vòng lặp Product (dưới Col) và chọn "Insert Directive..."



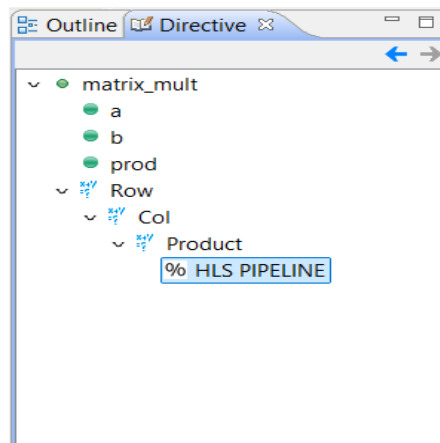
Hình 3B.11: Right-click vào Product để chèn directive

Trong Vivado HLS Directive Editor, chọn directive type là PIPELINE, để trống II (mặc định). Nhấn OK.



Hình 3B.12: Directive Editor – chọn PIPELINE cho vòng lặp Product

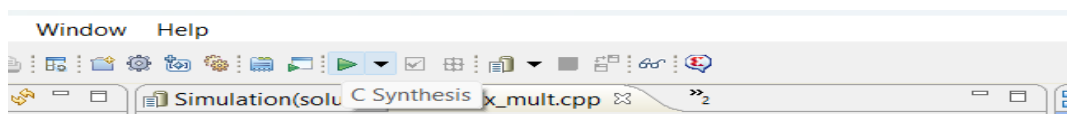
Sau khi thêm directive, tab Directive hiển thị "% HLS PIPELINE" được gán cho vòng lặp Product.



Hình 3B.13: Tab Directive sau khi thêm HLS PIPELINE cho Product

Bước 10: C Synthesis – Solution 2

Nhấn nút "C Synthesis" để tổng hợp solution2.



Hình 3B.14: Nút C Synthesis để chạy tổng hợp solution2

Kết quả Synthesis Report của solution2: Timing ước tính 7.664 ns (vượt target 5.00 ns). Latency tổng là 401 cycles (cải thiện so với 686). Vòng lặp Row_Col: latency 400, trip count 25, chưa pipelined. Vòng lặp Product: latency 11, II đạt được là 2 (không đạt target 1 do loop dependency), đã pipelined.

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	5.00	7.664	0.62

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
401	401	401	401	none

Detail

Instance

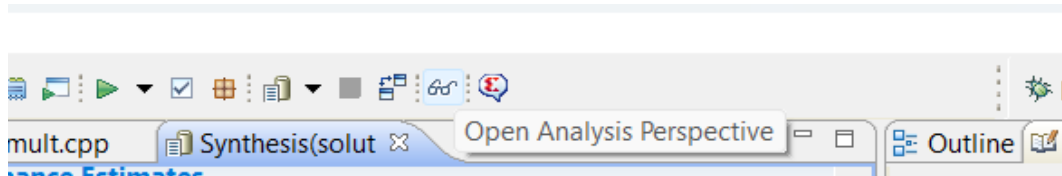
Loop

	Latency			Initiation Interval			
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- Row_Col	400	400	16	-	-	25	no
+ Product	11	11	4	2	1	5	yes

Hình 3B.15: Synthesis Report solution2 – Latency 401, Product pipelined với II=2

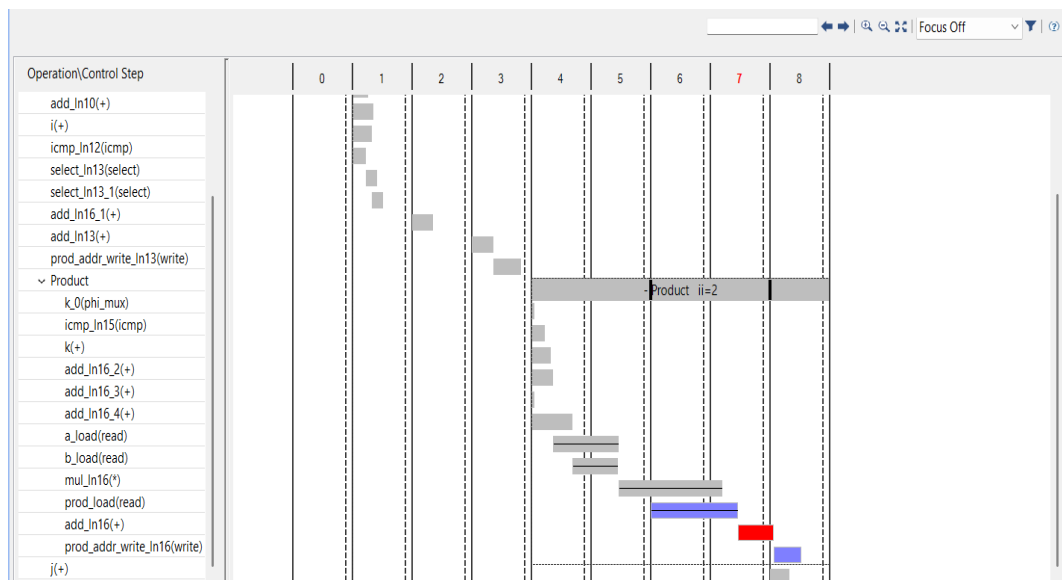
Bước 11: Phân tích qua Analysis Perspective

Nhấn nút "Open Analysis Perspective" để mở chế độ phân tích hiệu năng.



Hình 3B.16: Nút mở Analysis Perspective

Performance View của solution2 cho thấy biểu đồ lịch trình các thao tác theo clock cycles. Vòng lặp Product có II=2, với thanh màu đỏ tại clock 7 biểu thị xung đột dữ liệu (prod_addr_write), nguyên nhân khiến II không đạt target 1. Đây là vì phép toán $prod[i][j] += a[i][k] * b[k][j]$ vừa đọc vừa ghi vào array prod trên BRAM trong cùng một cycle.



Hình 3B.17: Performance View solution2 – Product ii=2, conflict tại clock 7

Bước 12: Cảnh báo II Violation

Console hiển thị cảnh báo WARNING: [SCHED 204-68] "The II Violation in module matrix_mult (Loop: Product): Unable to enforce a carried dependence between store operation (prod_addr_write_in16)

and load operation on array prod". Đây là do dependency giữa các iteration của vòng lặp Product khi truy cập BRAM.

```
WARNING: [SCHED 204-68] The II Violation in module 'matrix_mult' (Loop: Product): Unable to enforce a carried dependence between 'store' operation ('prod_addr_write_ln16', matrix_mult.cpp:16) of variable 'add_ln16', matrix_mult.cpp:16 on i
WARNING: [SCHED 204-68] The II Violation in module 'matrix_mult' (Loop: Product): Unable to enforce a carried dependence between 'store' operation ('prod_addr_write_ln16', matrix_mult.cpp:16) of variable 'add_ln16', matrix_mult.cpp:16 on i
WARNING: [SCHED 204-68] The II Violation in module 'matrix_mult' (Loop: Product): Unable to enforce a carried dependence between 'store' operation ('prod_addr_write_ln16', matrix_mult.cpp:16) of variable 'add_ln16', matrix_mult.cpp:16 on i
```

Hình 3B.18: Cảnh báo II Violation do loop dependency trên array prod

Bước 13: Xem source code gốc

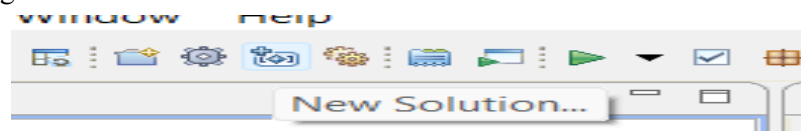
Source code tại dòng 15-17 của matrix_mult.cpp cho thấy vòng lặp Product sử dụng toán tử += (prod[i][j] += a[i][k] * b[k][j]), tạo ra dependency giữa các iteration do cần đọc và ghi prod trong cùng một chu kỳ.

```
13 prod[i][j] = 0;
14 // Do the inner product of a row of A and col of B
15 Product: for(int k = 0; k < IN_B_ROWS; k++) {
16     prod[i][j] += a[i][k] * b[k][j];
17 }
18 }
19 }
```

Hình 3B.19: Source code vòng lặp Product với phép += tạo ra loop dependency

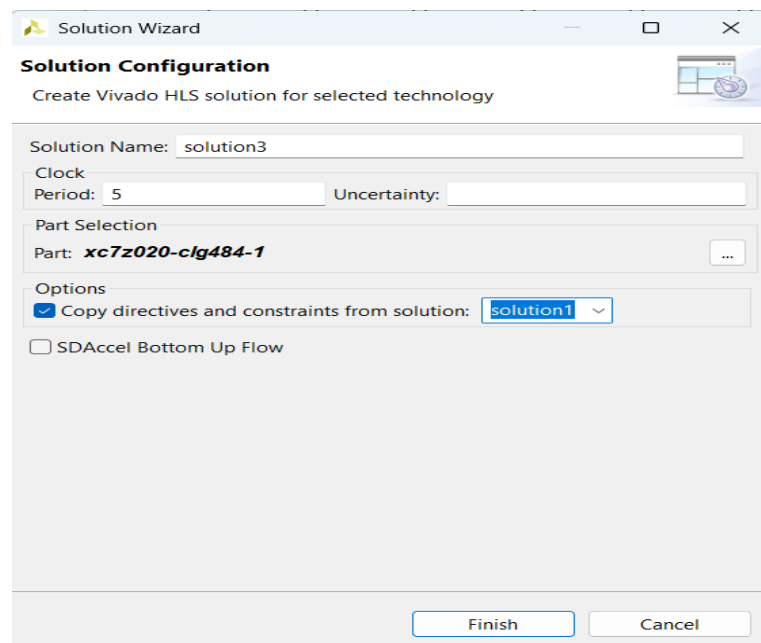
Bước 14: Tạo Solution 3 – Pipeline vòng lặp Col

Tạo solution3 bằng cách nhấn nút New Solution.



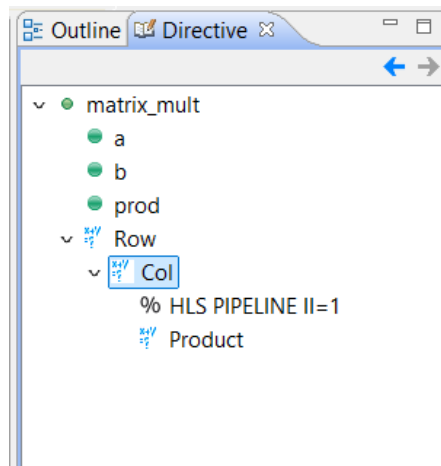
Hình 3B.20: Tạo solution mới

Trong Solution Wizard, đặt tên "solution3", copy directives từ solution1 (không có directive trước đó). Nhấn Finish.



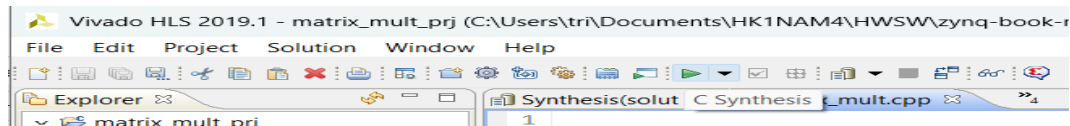
Hình 3B.21: Solution Wizard tạo solution3 từ solution1

Trong tab Directive, chèn directive PIPELINE II=1 vào vòng lặp Col (thay vì Product). Directive hiển thị "% HLS PIPELINE II=1" dưới Col.

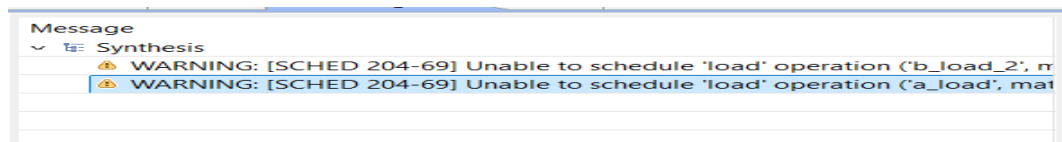


Hình 3B.22: Tab Directive solution3 – HLS PIPELINE II=1 cho vòng lặp Col

Nhấn C Synthesis. Console hiển thị cảnh báo SCHED 204-69: "Unable to schedule load operation (b_load_2) on array b due to limited memory ports" và tương tự cho a_load – không đủ memory ports do BRAM chỉ có 2 cổng.

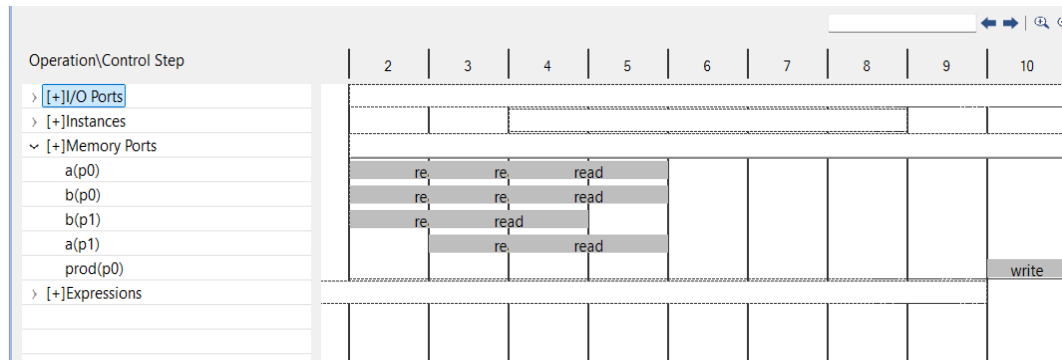


Hình 3B.23: Chạy C Synthesis cho solution3



Hình 3B.24: Cảnh báo – Unable to schedule load do limited memory ports

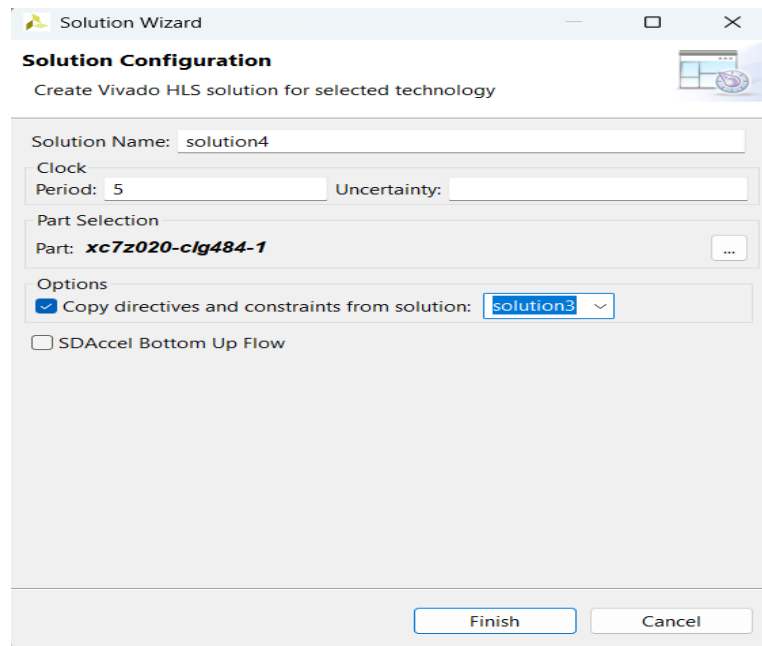
Chuyển sang Resource View trong Analysis Perspective. Memory Ports cho thấy các array a và b bị truy cập 3 lần cùng lúc (vượt quá 2 port của BRAM), dẫn đến không đạt II=1.



Hình 3B.25: Resource View solution3 – Memory ports bị quá tải (3 read operations cùng lúc)

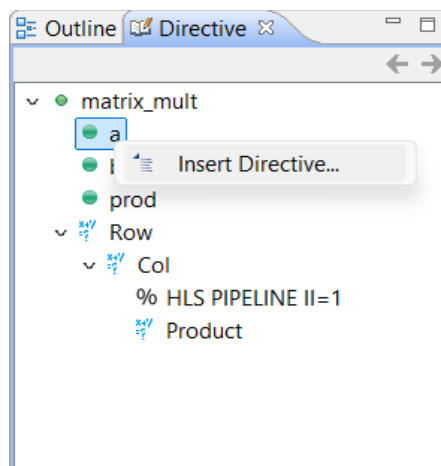
Bước 15: Tạo Solution 4 – Thêm ARRAY_RESHAPE

Tạo solution4 từ solution3 (copy directives). Nhấn Finish.



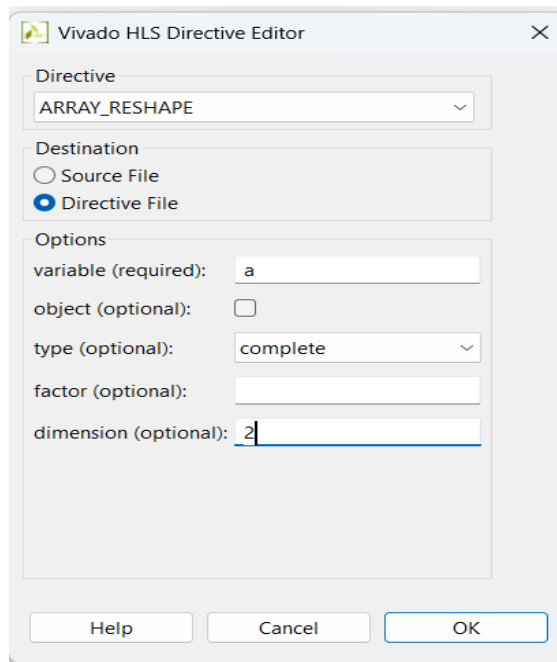
Hình 3B.26: Solution Wizard tạo solution4 từ solution3

Trong tab Directive, click chuột phải vào biến "a" và chọn "Insert Directive..."



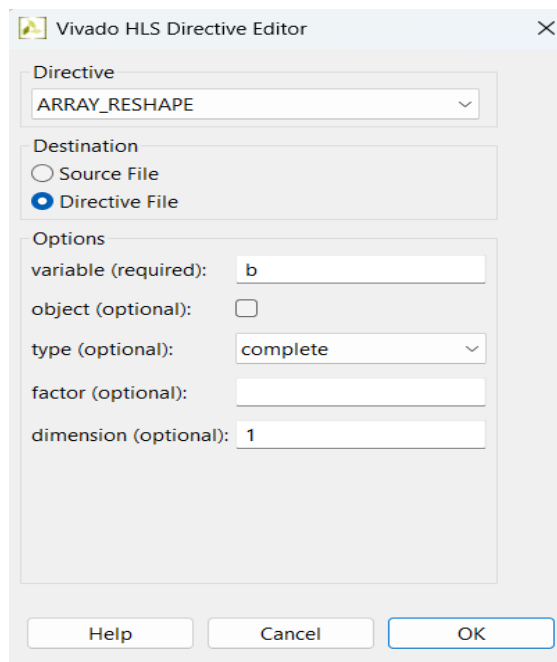
Hình 3B.27: Right-click biến a để thêm directive ARRAY_RESHAPE

Cấu hình directive ARRAY_RESHAPE cho biến a: type là "complete", dimension là 2 (theo chiều k của a[i][k]). Nhấn OK.



Hình 3B.28: Directive Editor – ARRAY_RESHAPE cho biến *a*, dimension=2

Tương tự, thêm directive ARRAY_RESHAPE cho biến *b* với dimension=1 (theo chiều *k* của $b[k][j]$). Nhấn OK.



Hình 3B.29: Directive Editor – ARRAY_RESHAPE cho biến *b*, dimension=1

Sau khi chạy C Synthesis, Synthesis Report của solution4 cho thấy: Timing ước tính 4.170 ns (đạt target 5.00 ns). Latency tổng là 33 cycles. Vòng lặp Row_Col: latency 31, II đạt được = 1 (đúng target), trip count 25, đã pipelined. Tài nguyên: 3 DSP48E, 387 FF, 361 LUT – tăng so với trước nhưng hiệu năng tốt hơn nhiều.

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	5.00	4.170	0.62

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
33	33	33	33	none

Detail

Instance

Loop

	Latency		Initiation Interval				
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- Row_Col	31	31	8	1	1	25	yes

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT	URAM
DSP	-	3	-	-	-
Expression	-	0	0	190	-
FIFO	-	-	-	-	-
Instance	-	-	-	-	-
Memory	-	-	-	-	-
Multiplexer	-	-	-	75	-
Register	0	-	387	96	-
Total	0	3	387	361	0
Available	280	220	106400	53200	0
Utilization (%)	0	1	~0	~0	0

Detail

Instance

DSP48E

Memory

FIFO

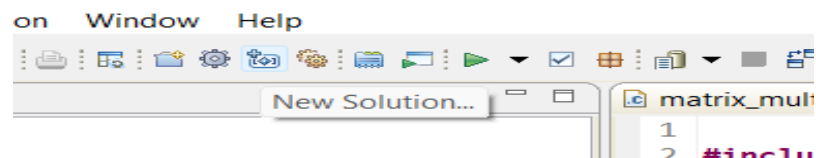
Expression

Multiplexer

Hình 3B.30: Synthesis Report solution4 – II=1 đạt được, latency chỉ còn 33 cycles

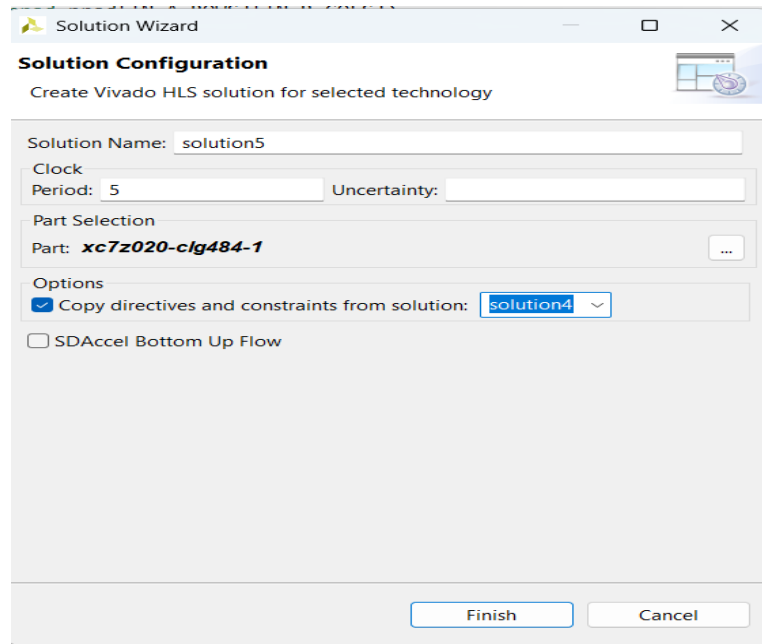
Bước 16: Tạo Solution 5 – Pipeline toàn bộ hàm

Tạo solution5 từ solution4. Nhấn nút New Solution.



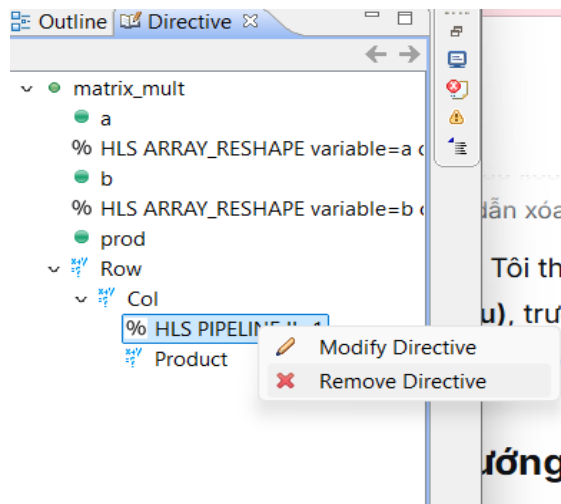
Hình 3B.31: Tạo solution5

Trong Solution Wizard, đặt tên "solution5", copy directives từ solution4. Nhấn Finish.

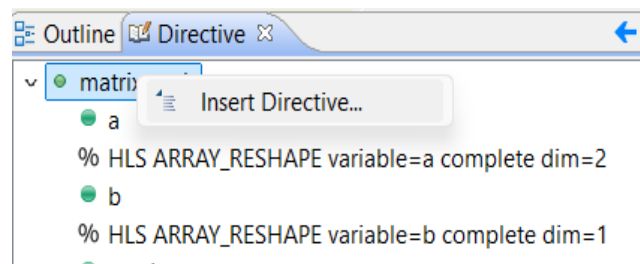


Hình 3B.32: Solution Wizard tạo solution5 từ solution4

Trong tab Directive: xóa bỏ directive PIPELINE II=1 trên Col bằng "Remove Directive", sau đó click chuột phải vào hàm `matrix_mult` (top level) và chọn "Insert Directive..." để thêm PIPELINE ở cấp hàm.

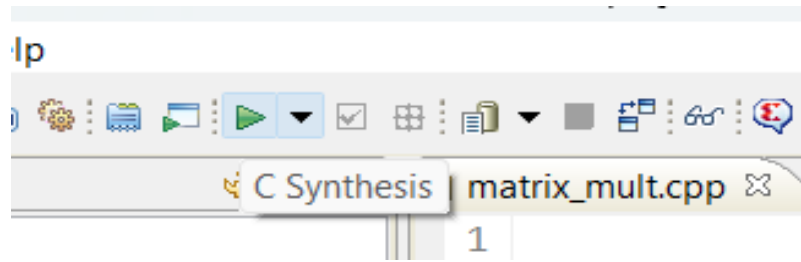


Hình 3B.33: Remove Directive PIPELINE trên Col



Hình 3B.34: Thêm PIPELINE ở cấp hàm `matrix_mult` (top level)

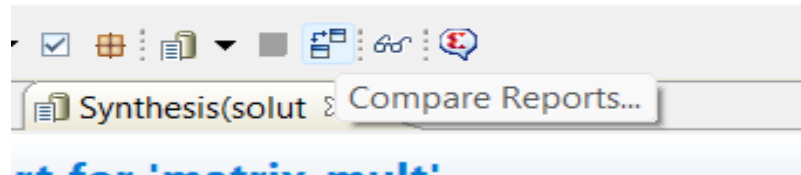
Nhấn C Synthesis để tổng hợp solution5.



Hình 3B.35: Chạy C Synthesis cho solution5

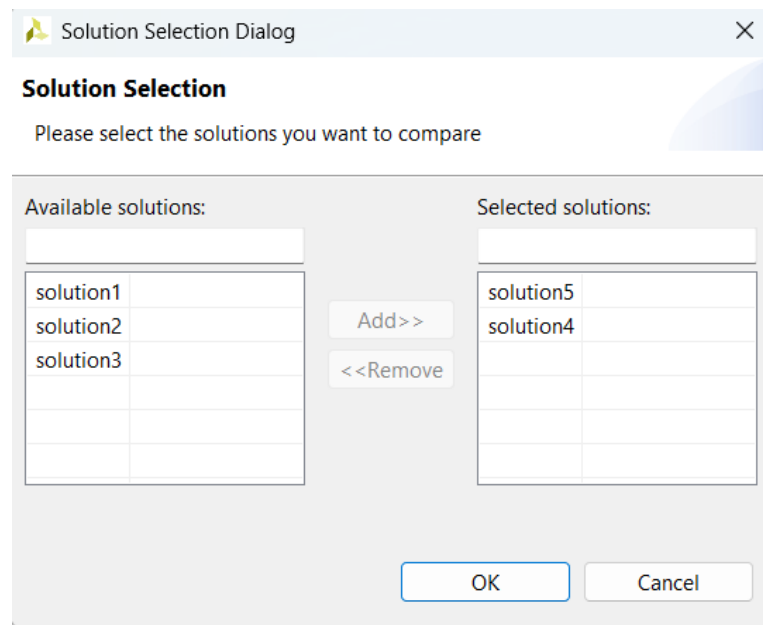
Bước 17: So sánh Solution 4 và Solution 5

Sử dụng tính năng Compare Reports (nhấn nút trên toolbar hoặc vào Project > Compare Reports).



Hình 3B.36: Nút Compare Reports trên toolbar

Trong Solution Selection Dialog, chọn solution4 và solution5 vào danh sách Selected Solutions. Nhấn OK để so sánh.



Hình 3B.37: Hộp thoại Solution Selection – chọn solution4 và solution5 để so sánh

Báo cáo so sánh Vivado HLS Report Comparison cho thấy: - Timing: cả hai đều Estimated 4.170 ns, đạt target 5.00 ns. - Latency: solution5 min/max = 23 cycles, solution4 = 33 cycles → solution5 nhanh hơn. - Interval: solution5 = 13, solution4 = 33 → solution5 có throughput cao hơn nhiều. - Tài nguyên: solution5 sử dụng 75 DSP48E, 3479 FF, 3008 LUT – lớn hơn rất nhiều so với solution4 (3 DSP48E, 387 FF, 361 LUT). Việc pipeline toàn bộ hàm yêu cầu unroll tất cả vòng lặp, làm tăng đáng kể tài nguyên phần cứng.

Vivado HLS Report Comparison			
All Compared Solutions			
solution5: xc7z020-clg484-1			
solution4: xc7z020-clg484-1			
Performance Estimates			
Timing (ns)			
Clock		solution5	solution4
ap_clk	Target	5.00	5.00
	Estimated	4.170	4.170
Latency (clock cycles)			
		solution5	solution4
Latency	min	23	33
	max	23	33
Interval	min	13	33
	max	13	33
Utilization Estimates			
		solution5	solution4
BRAM_18K	0	0	
DSP48E	75	3	
FF	3479	387	
LUT	3008	361	
URAM	0	0	
Resource Usage Implementation			
	solution5	solution4	
RTL	verilog	verilog	
SLICE	-	-	
LUT	-	-	
FF	-	-	
DSP	-	-	
SRL	-	-	
BRAM	-	-	
Need to run vivado synthesis/implementation to populate the real data for "-"			
Final Timing Implementation			
	solution5	solution4	
ap_clk	4.170	4.170	

Hình 3B.38: Report Comparison – solution5 vs solution4: latency thấp hơn nhưng tài nguyên cao hơn nhiều

PHẦN THỰC HÀNH 3C – TỔNG HỢP GIAO DIỆN (INTERFACE SYNTHESIS)

Phần này tìm hiểu cách Vivado HLS tự động tổng hợp các cổng giao diện RTL từ code C++. Nhóm khảo sát các giao thức ap_ctrl_hs và ap_memory được tạo ra cho hàm matrix_mult.

Bước 1: Chạy Tcl Script cho tut3C

Mở Vivado HLS Command Prompt, chuyển đến thư mục tut3C và chạy lệnh "vivado_hls -f run_hls_zed.tcl" để tạo project và chạy C Simulation cho bài tập 3C.

```
C:\Xilinx\Vivado\2019.1>cd C:\Users\tri\Documents\HK1NAM4\HWSW\project\project_3\hls\tut3C
C:\Users\tri\Documents\HK1NAM4\HWSW\project\project_3\hls\tut3C>vivado_hls -f run_hls_zed.tcl
```

Hình 3C.1: Chạy lệnh vivado_hls -f run_hls_zed.tcl trong thư mục tut3C

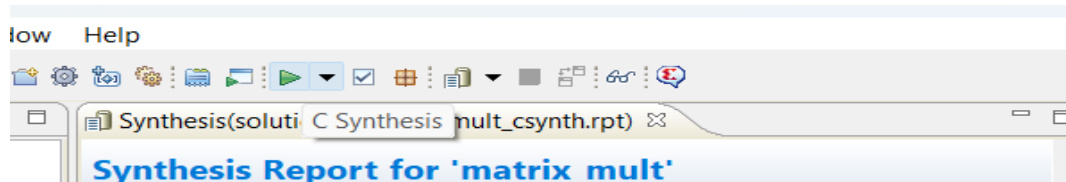
Kết quả C Simulation hiển thị "Test passed!" xác nhận code hoạt động đúng trước khi tổng hợp.

```
Test passed!
INFO: [SIM 211-1] CSim done with 0 errors.
INFO: [SIM 211-3] ***** CSIM finish *****
INFO: [Common 17-206] Exiting vivado_hls at Wed May 27 10:12:05 2026...
C:\Users\tri\Documents\HK1NAM4\HWSW\project\project_3\hls\tut3C>
```

Hình 3C.2: Kết quả "Test passed!" của C Simulation trong tut3C

Bước 2: Chạy C Synthesis và xem Interface Summary

Mở project trong Vivado HLS GUI, sau đó nhấn nút C Synthesis để tổng hợp RTL.



Hình 3C.3: Chạy C Synthesis để tổng hợp RTL cho tut3C

Bước 3: Phân tích Interface Summary

Sau khi tổng hợp, mở Synthesis Report và cuộn xuống phần Interface. Bảng Interface Summary liệt kê các RTL ports được tạo tự động:

- Giao thức ap_ctrl_hs (Block-level handshaking): ap_clk, ap_rst, ap_start, ap_done, ap_idle, ap_ready – dùng để điều khiển khởi động và kết thúc của hàm.
- Giao thức ap_memory (Array interface): Các array a, b, prod được ánh xạ thành các tín hiệu memory:
 - a: a_address0 (5-bit, out), a_ce0 (clock enable), a_q0 (8-bit, in – dữ liệu đọc)
 - b: b_address0 (5-bit, out), b_ce0 (clock enable), b_q0 (8-bit, in) - prod: prod_address0 (5-bit, out), prod_ce0, prod_we0 (write enable), prod_d0 (16-bit, out – dữ liệu ghi). Giao thức ap_memory phù hợp với cấu trúc BRAM (Block RAM) của FPGA. Array a và b là 8-bit (input), array prod là 16-bit (output) do kết quả phép nhân.

Interface					
Summary					
RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	matrix_mult	return value
ap_rst	in	1	ap_ctrl_hs	matrix_mult	return value
ap_start	in	1	ap_ctrl_hs	matrix_mult	return value
ap_done	out	1	ap_ctrl_hs	matrix_mult	return value
ap_idle	out	1	ap_ctrl_hs	matrix_mult	return value
ap_ready	out	1	ap_ctrl_hs	matrix_mult	return value
a_address0	out	5	ap_memory	a	array
a_ce0	out	1	ap_memory	a	array
a_q0	in	8	ap_memory	a	array
b_address0	out	5	ap_memory	b	array
b_ce0	out	1	ap_memory	b	array
b_q0	in	8	ap_memory	b	array
prod_address0	out	5	ap_memory	prod	array
prod_ce0	out	1	ap_memory	prod	array
prod_we0	out	1	ap_memory	prod	array
prod_d0	out	16	ap_memory	prod	array

Hình 3C.4: Interface Summary – các RTL ports được tổng hợp tự động với giao thức ap_ctrl_hs và ap_memory

KẾT LUẬN

Qua bài thực hành Lab 3, nhóm đã nắm được quy trình thiết kế với Vivado HLS từ đầu đến cuối:

- Phần 3A: Tạo project HLS thành công bằng cả hai phương pháp GUI và Tcl Script. C Simulation "Test passed!" xác nhận code đúng.
- Phần 3B: Thực hiện tối ưu hóa qua 5 solution. Solution 1 (không tối ưu): latency 686 cycles. Solution 2 (PIPELINE Product): latency 401, II=2. Solution 3 (PIPELINE Col): cảnh báo memory port. Solution 4 (PIPELINE Col + ARRAY_RESHAPE a,b): đạt II=1, latency 33 cycles – tối ưu nhất về cân bằng hiệu năng/tài nguyên. Solution 5 (PIPELINE hàm): latency 23, interval 13 nhưng tốn 75 DSP.
- Phần 3C: Hiểu rõ cách Vivado HLS tự động sinh giao diện RTL: giao thức ap_ctrl_hs cho điều khiển block-level và ap_memory cho các array tương thích BRAM.

Hình ảnh nhóm:

